
PERTEMUAN 1

TEKNOLOGI IOT II



Disusun oleh :

Nama : syafa sartika putri aryanti

NPM : 25782056

Kelas : TRI-3B

Dosen:

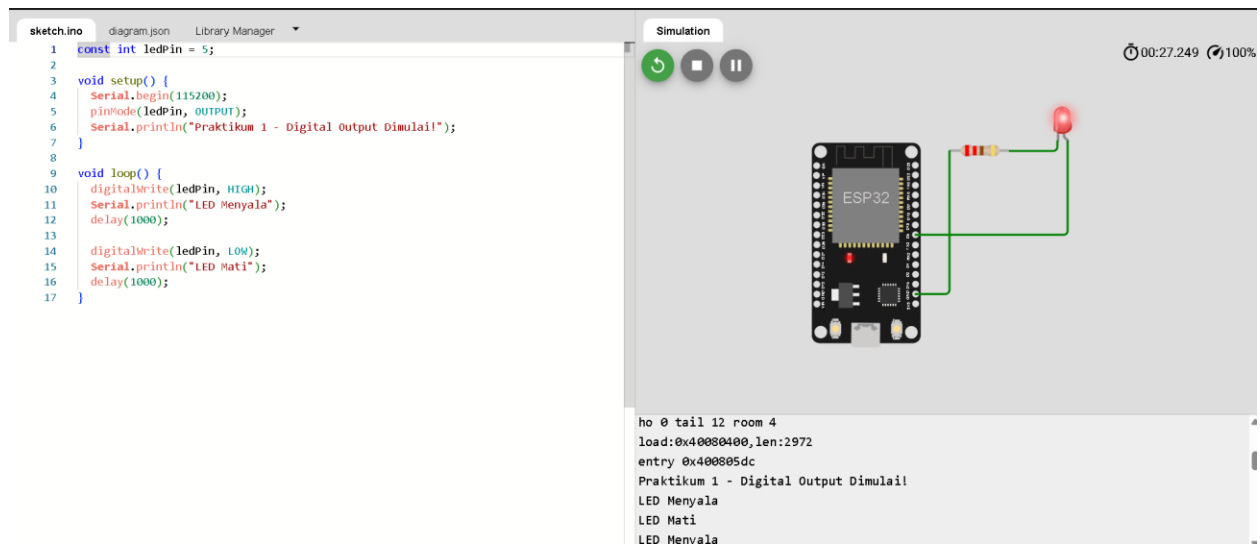
Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

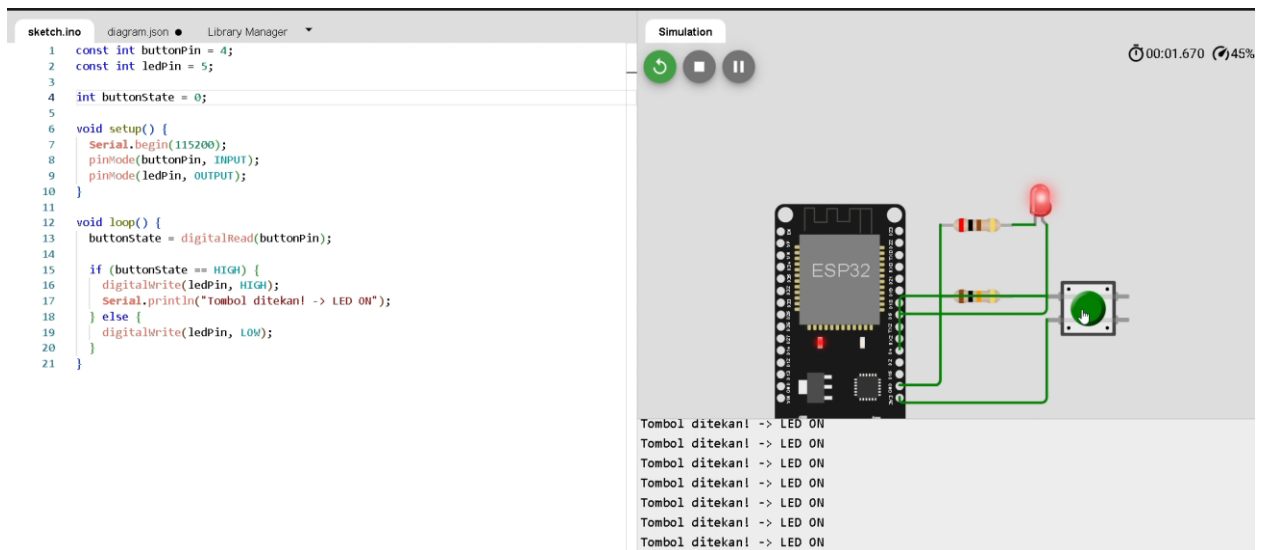
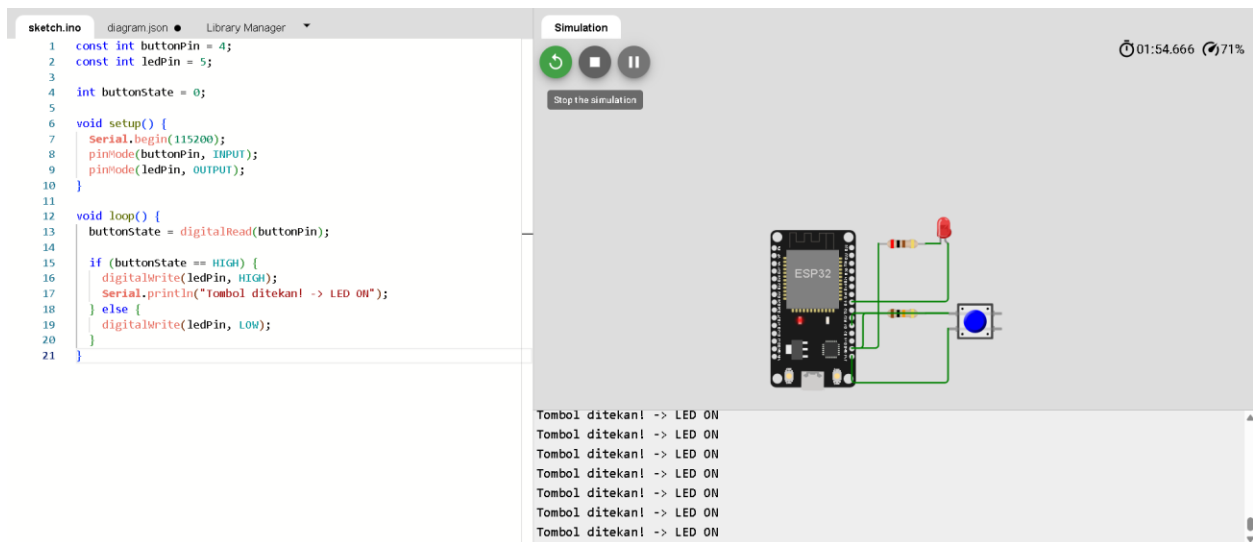
2026/2027

Praktikum 1: *Digital Output* (Menyalakan LED)



Percobaan ini bertujuan mempraktikkan pengontrolan sinyal digital keluaran (*digital output*) pada mikrokontroler untuk mengoperasikan LED. Di dalam baris kode `setup()`, pin GPIO 5 dikonfigurasi sebagai jalur OUTPUT serta jalur komunikasi *Serial Monitor* diaktifkan pada kecepatan 115200 baud. Selanjutnya pada fungsi `loop()`, mikrokontroler mengalirkan sinyal logika HIGH yang membuat LED menyala dan secara bersamaan mencetak notifikasi "LED Menyala" ke layar *Serial Monitor*. Setelah tertahan selama interval 1 detik (`delay(1000)`), sinyal diubah menjadi LOW yang mengakibatkan LED padam disertai pengiriman pesan "LED Mati". Siklus ini dieksekusi berulang kali secara terus-menerus, memicu efek LED berkedip secara konstan sekaligus membuktikan bahwa seluruh rangkaian dan logika program telah berfungsi dengan baik.

Praktikum 2: *Digital Input (Membaca Push Button)*



Percobaan ini dirancang untuk membaca interaksi tombol tekan (*push button*) sebagai *digital input* menggunakan bantuan konfigurasi resistor *pull-down* eksternal. Di dalam fungsi `setup()`, pin saklar (GPIO 4) ditetapkan sebagai INPUT, pin LED (GPIO 5) sebagai OUTPUT, dan komunikasi serial diaktifkan pada kecepatan 115200 baud. Pada bagian `loop()`, mikrokontroler memantau kondisi tegangan pin input secara *real-time* memanfaatkan instruksi `digitalRead(buttonPin)`.

Saat saklar ditekan, arus 3,3V mengalir ke pin GPIO 4 sehingga kondisi terbaca HIGH. Situasi ini memicu instruksi *conditional* `if` untuk memberi luaran HIGH ke pin LED sehingga LED aktif menyala, disertai pencetakan notifikasi "Tombol ditekan! -> LED ON" pada *Serial Monitor*. Begitu saklar dilepas, resistor 10 kΩ menarik tegangan kembali ke jalur *Ground* (kondisi LOW),

yang mana blok else akan memadamkan LED. Hasil pengujian ini menunjukkan bahwa sistem berhasil membaca masukan fisik dan meresponsnya sesuai logika program.

1.6 Latihan

1) Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Penjelasan:

- **Syarat Booting Pin D8 (GPIO 15):** Menurut tabel referensi *pinout*, pin GPIO 15 harus berada dalam kondisi **LOW** saat proses nyala awal (*booting*) agar mikrokontroler dapat masuk ke mode operasi normal.
- **Dampak Menekan Tombol:** Pada konfigurasi *Active-High (pull-down)*, menekan tombol akan mengalirkan tegangan 3,3V secara langsung ke pin D8. Hal ini memaksa kondisi pin D8 menjadi **HIGH** di detik awal papan dinyalakan.
- **Akibatnya:** Kondisi **HIGH** pada D8 saat *booting* memicu kegagalan sistem pengapusan/pembacaan memori bawaan, sehingga NodeMCU akan *stuck* (macet), terus melakukan *restart* berulang kali (*boot loop*), atau program tidak akan berjalan sama sekali sampai tombol dilepaskan.

- 2) **Latihan 2 (Implementasi Kode):** Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.



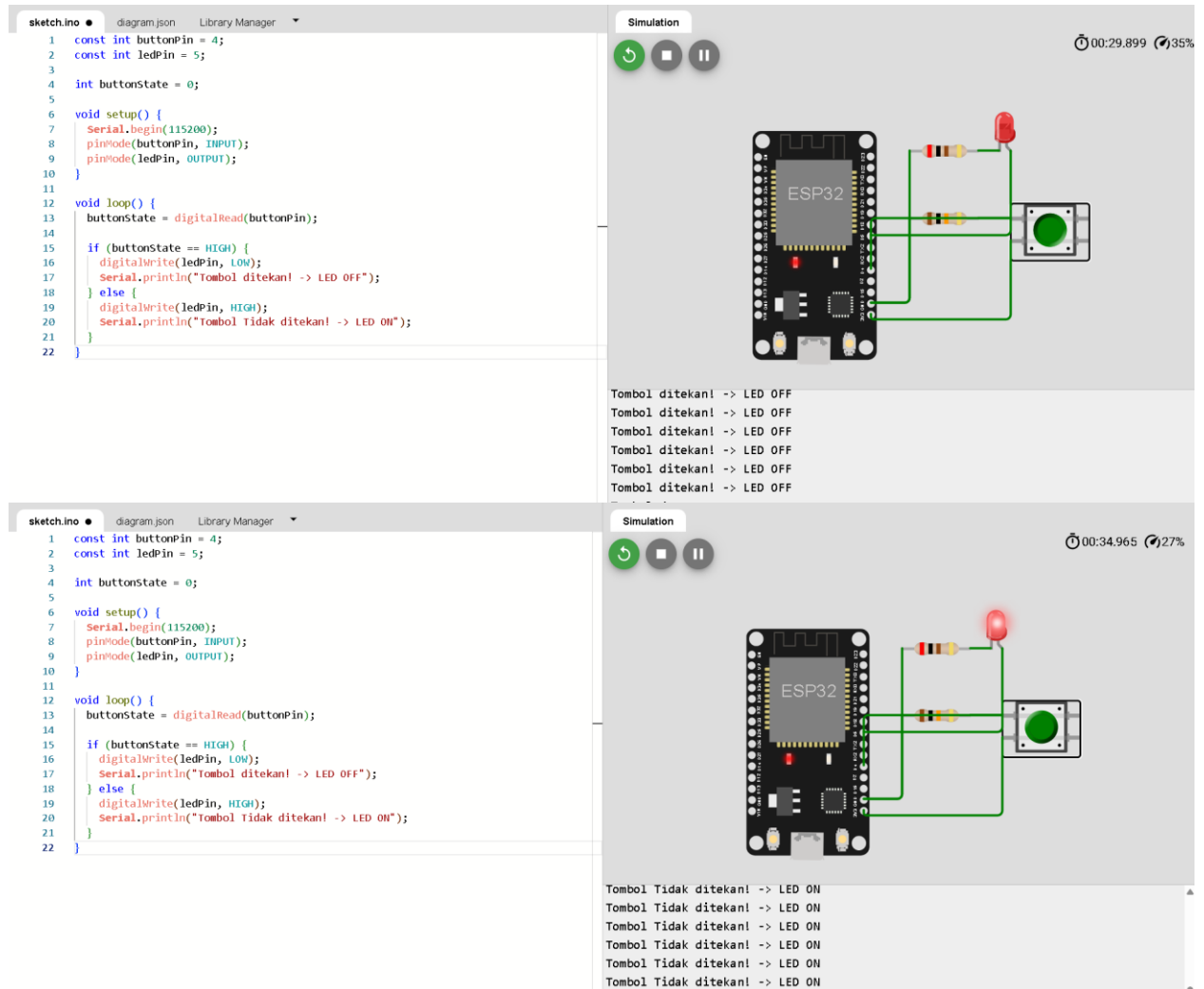
Berdasarkan hasil simulasi, LED menyala sangat cepat selama 200 ms dan mati selama 800 ms secara berulang. Teks status pada Serial Monitor juga mencetak "LED Menyala" dan "LED Mati" selaras dengan jeda waktu tersebut.

- 3) **Latihan 3 (Analisis Sirkuit):** Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Apakah program *error* kompilasi? Tidak, program tetap dapat dikompilasi karena masalah tersebut terjadi pada rangkaian *hardware*, bukan sintaks kode program.

Mengapa LED tidak menyala? LED tidak menyala karena dipasang terbalik (kondisi *reverse bias*). LED hanya mengalirkan arus satu arah (Anoda ke Katoda), sehingga ketika dipasang terbalik, arus listrik terhambat dan tidak dapat mengalir untuk menyalakan LED.

4) Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru mati."



Kondisi if-else diubah dengan memeriksa kondisi `buttonState == LOW` terlebih dahulu. Ketika tombol tidak ditekan (LOW), program mengeksekusi `digitalWrite(ledPin, HIGH)` sehingga LED menyala. Sebaliknya, saat tombol ditekan (HIGH), blok `else` mengeksekusi `digitalWrite(ledPin, LOW)` sehingga LED padam. Perubahan ini berhasil membalik perilaku kontrol LED sesuai instruksi soal.

1.7 Tugas

Modifikasi Sistem Sakelar *Toggle (Latching)*

The image displays two screenshots of an Arduino IDE simulation environment, showing the code and the corresponding hardware simulation.

Top Screenshot:

- Code (sketch.ino):**

```
1 const int buttonPin = 4;
2 const int ledPin = 5;
3
4 int buttonState = LOW;
5 int lastButtonState = LOW;
6 bool ledState = false;
7
8 void setup() {
9   Serial.begin(115200);
10  pinMode(buttonPin, INPUT);
11  pinMode(ledPin, OUTPUT);
12  digitalWrite(ledPin, LOW);
13  Serial.println("Sistem Toggle Sakelar Siap!");
14 }
15
16 void loop() {
17   buttonState = digitalRead(buttonPin);
18
19   if (buttonState == HIGH && lastButtonState == LOW) {
20     ledState = !ledState;
21     digitalWrite(ledPin, ledState ? HIGH : LOW);
22
23     if (ledState) {
24       Serial.println("Tombol ditekan -> LED ON");
25     } else {
26       Serial.println("Tombol ditekan -> LED OFF");
27     }
28     delay(200);
29   }
30
31   lastButtonState = buttonState;
32 }
```
- Simulation:** Shows an ESP32 microcontroller connected to a button and an LED. The LED is currently ON (red).
- Serial Monitor:** Displays the following output:

```
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Sistem Toggle Sakelar Siap!
Tombol ditekan -> LED ON
```

Bottom Screenshot:

- Code (sketch.ino):** The code is identical to the top screenshot.
- Simulation:** Shows the same hardware setup. The LED is currently OFF (green).
- Serial Monitor:** Displays the following output:

```
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
```

Pada tahapan inisialisasi, kondisi awal LED disetel dalam keadaan padam. Ketika penekanan tombol dideteksi, instruksi program secara otomatis mengubah status luaran LED serta menahannya pada kondisi tersebut meskipun input tombol telah dilepaskan. Pemrosesan ini memanfaatkan variabel pelacak status (*state tracking*) untuk menyimpan logika terkini, disertai penambahan durasi `delay(200)` ms sebagai mekanisme *software debouncing* guna memfilter pemicu ganda akibat pantulan mekanis. Melalui skema ini, setiap satu kali aksi penekanan tombol akan menghasilkan transisi status LED secara konsisten, baik dari **OFF** → **ON** maupun dari **ON** → **OFF**.

